

Contents

1	Introduction	2
2	Neural Processing Unit (NPU) and Neural Engines	2
2.1	What is an NPU?	2
2.2	Architecture and Operation	2
2.3	Applications	3
2.4	Advantages and Limitations	3
2.5	Future Trends	4
3	Unified CPU, GPU and RAM Architectures	4
3.1	Traditional Architecture	4
3.2	Unified Memory Architecture	4
3.3	CPU-GPU Integration	4
3.4	Performance Benefits	5
3.5	Limitations	5

1 Introduction

2 Neural Processing Unit (NPU) and Neural Engines

2.1 What is an NPU?

Neural Processing Unit (NPU) is a processing unit like CPU and a GPU but specialized for Artificial Intelligence and Machine Learning Tasks. CPU designed to handle many different general purpose tasks and GPU mainly designed to perform graphics and parallel calculation when the NPU is mainly designed to perform mathematical operations used by neural networks such as, working with vectors and tensors. NPU is not a unit that replaces the CPU and the GPU; instead, they work together and each processor handles the type of work it is best suited for (heterogeneous computing). In modern devices such as smart phones , laptops and other smart electronic devices , the NPU is usually integrated with CPU and GPU inside a single System on Chip (Soc) to perform Artificial Intelligence tasks such as face recognition, speech recognition and image enhancement ect. Since Neural Processing Units (NPU) are specialized for AI workloads, they can provide high performance with relatively lower power consumption than CPUs and GPUs.

2.2 Architecture and Operation

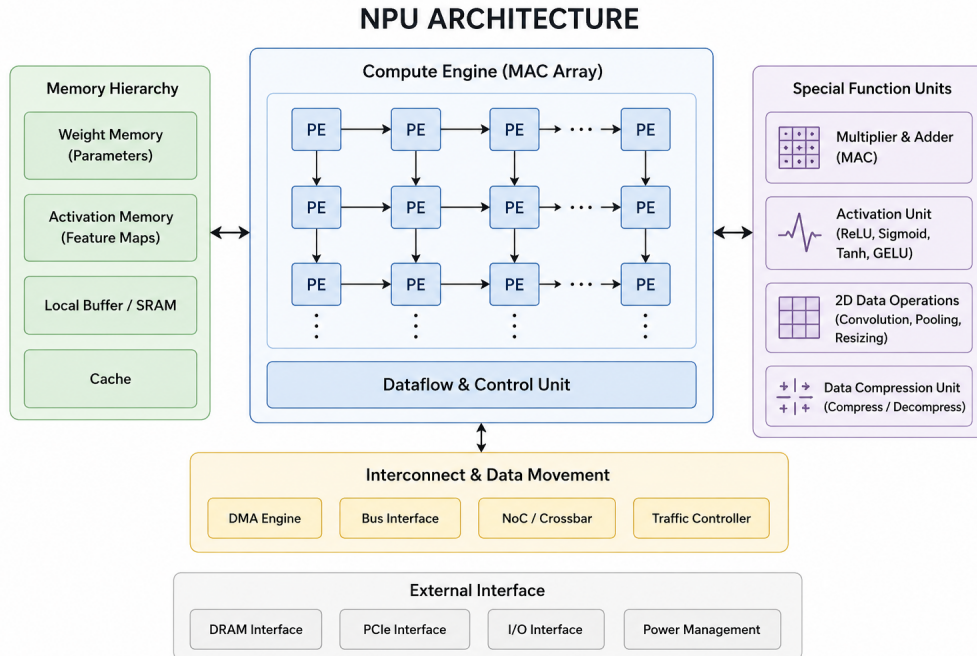


Figure 1: Simplified NPU Architecture

The Neural Processing Unit architecture is the internal organization of specialized hardware that is designed to perform neural network operations. A neural-network accelerator (NPU) contains

a compute engine with many Processing Elements (PEs) that perform small calculations such as multiplications and accumulation (MAC), which are the fundamentals of neural-network calculations. Research on the Eyeriss accelerator demonstrates how parallel processing elements can be organized to efficiently execute convolutional neural networks while reducing energy consumption [1]. Similarly, Google’s TPU (Tensor Processing Unit) uses a large matrix of MAC units as its main computational engine. That shows the high importance of parallel computation in AI accelerators [2]. NPUs require an efficient memory hierarchy to store neural-network weights, input data, and intermediate results. Also, keeping frequently used data close to the processing unit reduces expensive data movements and improves energy efficiency [1]. In addition, the data flow and interconnect mechanism control how data moves between the memory and the processing elements. Eyeriss v2, for example, uses a flexible on-chip network to adapt data movement to different neural-network workloads [3]. Therefore, the NPU architecture mainly combines parallel processing elements, MAC units, local memory, data movement networks, and control mechanisms to achieve high performance and energy-efficient AI processing [1–3].

2.3 Applications

Neural Processing Units are mainly used to improve Artificial Intelligence and Machine Learning tasks. One important application of NPUs is Smartphones. Smartphones use NPUs for image enhancements, face recognition, speech processing, and other AI-based features with low power consumption [4]. NPUs are also useful in computer vision, including real-time image processing, object detection and image classification [1]. In IoT and edge devices, they can use sensors and AI data locally by reducing latency and dependence on cloud servers [5]. NPUs can support healthcare, robotics, autonomous systems and smart environments, where fast AI interfaces are required [5]. As AI models become more common on modern edge devices, specialized neural-network accelerators are more important for achieving real-time performance within limited power and computing resources [5].

2.4 Advantages and Limitations

NPUs provide high performance and energy efficiency for AI work since they use specialized hardware and massive parallel processing for neural-network operations [6]. They can reduce data movement and reuse data locally, which helps to lower power consumption and improve processing efficiency [6]. NPUs are especially useful for real-time AI applications on mobile and edge devices where performance and battery life are important [7]. There are some limitations also. NPU may be less flexible than a CPU or GPU when running different or rapidly changing neural-network models [7], [8]. Because some accelerator designs work best only with specific model characteristics, such as particular data patterns or sparsity [8]. In addition, specialized hardware can require greater design complexity and become less efficient when the workload does not match its architecture [7]. Therefore, NPUs provide excellent AI performance per watt, but this advantage comes with trade-offs in flexibility, programmability, and workload compatibility.

2.5 Future Trends

In the future NPU development is expected to focus on higher energy efficiency, greater parallelism, and improved support for increasingly complex AI models [9]. NPUs are becoming more important in edge AI, allowing devices such as smartphones, IoT systems, robots and vehicles to process AI locally with low latency [10]. Researchers are exploring quantization, sparsity, model compression and memory-centric architecture to reduce the computation and data movement cost [9], [11]. Another emerging direction is on-device learning, which allows AI models to adapt directly to changing environments [12]. However, future NPUs face challenges including limited memory, high data movement cost, hardware complexity, model compatibility, scalability, and power constraints [9], [11]. As a result of that, future NPU research will increasingly focus on hardware-software co-design, flexible architectures, efficient memory systems and specialized support for emerging AI workloads.

3 Unified CPU, GPU and RAM Architectures

3.1 Traditional Architecture

In traditional computer architecture, the CPU and the GPU operate as separate processing units with different memory resources. The CPU normally access the system RAM, while the GPU has its own high-bandwidth memory. So the data that is required by the GPU must be transferred from CPU memory to the GPU memory through an interconnection such as PCI Express. After the GPU processing finishes, the processed data should be copied back to system memory [1]. This separation provides high GPU bandwidth, but it introduces data-transfer overhead and increases programming complexity.

3.2 Unified Memory Architecture

Unified Memory Architecture (UMA) reduces this separation by allowing the CPU and the GPU to access a common memory. Instead of maintaining completely separate memory, both processors can operate on data located in shared system memory. Heterogeneous System Architecture (HSA) extends this concept by providing shared virtual memory addressing and mechanisms for cooperation between heterogeneous processing units [2]. A modern unified memory system can automatically manage memory placement and migration, reducing the need for programmers to explicitly copy data between CPU and GPU memories [3].

3.3 CPU-GPU Integration

CPU-GPU integration places the CPU cores and the GPU compute units within the same package (System-on-Chip). Integrated design reduces the physical communication distance and can provide a more tightly coupled processing environment. Heterogeneous System Architecture-based architectures allow different processing units to cooperate through shared memory and a common programming model [2]. Modern processors such as Apple silicon demonstrate this approach by combining CPU, GPU, and other accelerators around a unified memory architecture [4].

3.4 Performance Benefits

Unified architectures can improve performance by cutting down on data movement between different memory pools. When data doesn't have to move forth as much communication overhead goes down. This helps CPUs and GPUs work together better especially when the workload involves data exchanges. With shared memory both the CPU and GPU can use the physical memory space. That means memory is used efficiently since there's no need to duplicate data across separate systems. Also integrated designs often save energy because data doesn't always have to travel over an external connection [1] [2]. These benefits make unified architectures very useful for applications, in intelligence, graphics, multimedia processing and scientific computing.

3.5 Limitations

Unified architectures have some drawbacks even though they bring benefits. The CPU and GPU share the memory bandwidth. When both processors try to read or write amounts of data at the same time they can get blocked. That blockage is called contention. Also when a unified-memory system moves data from one place to another or when a page fault happens extra delay can appear. In some cases this delay can slow down the system. Moreover the shared memory pool may not give the GPU much fast high-bandwidth memory as a separate GPU that has its own dedicated VRAM. So while unified architectures make programming easier and help data flow between CPU and GPU the speed of the system depends a lot, on how the memory can move data on what the work looks like and on how the hardware was built.

References

- [1] Y.-H. Chen, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, 2017.
- [2] N. P. Jouppi, "In-datacenter performance analysis of a tensor processing unit," *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017.
- [3] J. E. Y.-H. Chen T.-J. Yang and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.
- [4] Y.-H. C. et al, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.
- [5] M. A. et al., "Embedded deep learning accelerators: A survey on recent advances," *IEEE Transactions on Artificial Intelligence*, vol. vol. 5, no. no. 5, 2024.
- [6] J. S. E. Y.-H. Chen T. Krishna and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. vol. 52, no. no. 1, Jan. 2017.

- [7] J. E. Y.-H. Chen T.-J. Yang and V. Sze, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. vol. 9, no. no. 2, 2019.
- [8] J. E. Y.-H. Chen T.-J. Yang and V. Sze, “Understanding the limitations of existing energy-efficient design approaches for deep neural networks,” in *Proc. SysML*, 2018.
- [9] T. Mohaidat and K. Khalil, “A survey on neural network hardware accelerators,” *IEEE Transactions on Artificial Intelligence*, vol. vol. 5, 2024.
- [10] T. M. et al, “Revisiting edge ai: Opportunities and challenges,” *IEEE Internet Computing*, vol. vol. 28, no. no. 4, 2024.
- [11] Y.-H. C. et al, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. vol. 9, no. no. 2, 2019.
- [12] J. Z. et al, “Dq-stp: An efficient sparse on-device training processor based on low-rank decomposition and quantization for dnn,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. vol. 71, no. no. 4, 2024.